

Week 6 - Spark Assignment

Apache Spark Fundamentals

Internship: Celebal Technologies - Data Engineering Internship

Objective

The objective of this assignment is to understand Apache Spark architecture, Spark execution model, DataFrame operations, transformations, actions, schema handling, and efficient processing of CSV and Parquet files using PySpark.

Q1. Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Answer

Apache Spark follows a master-worker architecture where different components work together to execute distributed data processing tasks.

Driver

- The Driver is the main process of a Spark application.
- It creates the Spark Session and coordinates the execution of the application.
- It converts user code into execution plans (DAG) and schedules tasks for executors.
- It collects the results returned by executors.

Cluster Manager

- The Cluster Manager manages the computing resources of the cluster.
- It allocates CPU cores and memory to Spark applications.
- It launches and monitors executor processes on worker nodes.
- Examples include Standalone Cluster Manager, YARN, Kubernetes, and Apache Mesos.

Executor

- Executors are worker processes that run on cluster nodes.
- They execute the tasks assigned by the Driver.
- They perform computations and store intermediate data in memory or disk.
- After execution, they send the results back to the Driver.

Summary

Component	Responsibility
Driver	Controls the Spark application and schedules tasks
Cluster Manager	Allocates cluster resources and manages executors
Executor	Executes tasks and processes data

Q2. How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Answer

Lazy Evaluation is one of the most important optimization techniques in Apache Spark. Instead of executing each transformation immediately, Spark records all transformations and waits until an **Action** (such as `show()`, `collect()`, or `count()`) is called.

When an action is triggered, Spark combines all the transformations into an optimized execution plan called a **Directed Acyclic Graph (DAG)**. This optimization minimizes unnecessary computations and improves the overall performance of data processing.

Advantages of Lazy Evaluation

- Delays execution until an action is invoked.
- Optimizes the execution plan using the DAG.
- Reduces unnecessary data movement and computations.
- Improves execution speed for large datasets.
- Efficiently utilizes cluster resources.

Example

Suppose we apply multiple transformations such as filtering, selecting columns, and renaming columns. Spark does not execute these operations one by one. Instead, it waits until an action like `show()` is called and then executes all transformations together in an optimized manner.

Spark Environment Setup

Before working with DataFrames, we need to import the required PySpark libraries and create a Spark Session. The Spark Session acts as the entry point for all Spark applications.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder \
    .appName("Week6_Spark_Assignment") \
```

```
.getOrCreate()

print("Spark Session Created Successfully!")
```

Q3. Read a CSV File

Problem Statement

Read the CSV file located at **data/source.csv**, ensuring that:

- The first row is treated as the header.
- Schema is automatically inferred.

Solution

The following PySpark code reads the CSV file into a DataFrame using `header=True` and `inferSchema=True`.

```
df = spark.read.csv(
    "../data/source.csv",
    header=True,
    inferSchema=True
)

df.show()

df.printSchema()
```

Q4. Difference Between CSV and Parquet

Answer

CSV (Comma-Separated Values) and Parquet are two commonly used file formats for storing data in Apache Spark. They differ significantly in how data is stored and accessed.

Feature	CSV	Parquet
Storage Format	Row-based	Columnar
File Size	Larger	Smaller (Compressed)
Read Performance	Slower	Faster
Compression	No built-in compression	Built-in compression
Schema Support	No	Yes
Predicate Pushdown	Not Supported	Supported

Why does it matter for performance?

- **CSV** stores complete rows together, so Spark must read the entire row even if only one column is required.

- **Parquet** stores data column by column, allowing Spark to read only the required columns.
- Because of its columnar storage and compression, Parquet reduces disk I/O, memory usage, and execution time, making it the preferred format for large-scale analytics.

Q5. Select product_id and price for Electronics Category

Problem Statement

Select only the **product_id** and **price** columns from the DataFrame where the **category** is **Electronics**.

Solution

The following query filters the DataFrame for the Electronics category and selects only the required columns.

```
df_q5 = df.filter(
    col("category") == "Electronics"
).select(
    "product_id",
    "price"
)
df_q5.show()
```

Q6. Rename Column and Cast Data Type

Problem Statement

Rename the column **old_name** to **new_name** and convert the **price** column from **String** to **Double**.

Solution

The `withColumnRenamed()` function is used to rename an existing column, while the `cast()` function is used to convert a column from one data type to another.

Since the CSV schema was inferred automatically, the **price** column was detected as an Integer. For demonstration purposes, it is first converted to **String** and then cast to **Double** to satisfy the assignment requirement.

```
# Rename old_name to new_name
df = df.withColumnRenamed("old_name", "new_name")

# Convert Integer -> String
df = df.withColumn("price", col("price").cast("string"))

# Convert String -> Double
df = df.withColumn("price", col("price").cast("double"))

df.select("new_name", "price").show()
```

```
df.printSchema()
```

Observation

- The column **old_name** was successfully renamed to **new_name**.
- The **price** column was successfully converted to the **Double** data type.
- These operations demonstrate Spark's ability to modify DataFrame schemas without altering the original data source.

Q7. Lineage Graph (DAG) and Fault Tolerance

Answer

Apache Spark maintains a **Lineage Graph**, also known as a **Directed Acyclic Graph (DAG)**, which records all the transformations applied to a DataFrame.

Instead of storing multiple copies of data, Spark keeps track of how the data was generated. If a worker node or executor fails, Spark uses the DAG to recompute only the lost partitions from the original data instead of recalculating the entire dataset.

Advantages of DAG

- Provides fault tolerance without replicating data.
- Recomputes only the missing partitions.
- Reduces recovery time after failures.
- Optimizes execution by creating an efficient execution plan.

Conclusion

The Lineage Graph enables Spark to recover lost data efficiently, making distributed data processing reliable and fault tolerant.

Q8. Filter Completed Orders

Problem Statement

Filter the DataFrame to display only those records where:

- **status = 'Completed'**
- **amount > 1000**

Solution

The following query uses the `filter()` function with multiple conditions joined by the logical **AND (&)** operator.

```
df_orders = df.filter(
```

```
(col("status") == "Completed") &
(col("amount") > 1000)
)
df_orders.show()
```

Observation

- Successfully filtered records where the order status is **Completed**.
- Only records with **amount greater than 1000** were displayed.
- Multiple conditions were combined using the logical **AND (&)** operator.

Q9. Predicate Pushdown in Parquet

Answer

Predicate Pushdown is an optimization technique used by Apache Spark when reading **Parquet** files. Instead of loading the entire dataset into memory, Spark pushes the filter conditions down to the Parquet storage layer.

As a result, only the rows that satisfy the filter condition are read from disk, while the remaining rows are skipped.

Advantages of Predicate Pushdown

- Reduces the amount of data read from disk.
- Minimizes memory usage.
- Decreases disk I/O operations.
- Improves query execution speed.
- Enhances overall performance when working with large datasets.

Conclusion

Predicate Pushdown makes Parquet highly efficient for analytical workloads because Spark reads only the required data instead of scanning the entire file.

Q10. Add a New Column `final_price`

Problem Statement

Create a new column named **final_price** by multiplying the **base_price** column by **1.18**, representing an 18% tax.

Solution

The `withColumn()` function is used to create a new column. The new column is calculated by multiplying the existing `base_price` column by `1.18`.

```
df = df.withColumn(
    "final_price",
    col("base_price") * 1.18
)

df.select(
    "new_name",
    "base_price",
    "final_price"
).show()
```

Observation

- A new column **final_price** was successfully created.
- The values were calculated using the formula:
$$\text{final_price} = \text{base_price} \times 1.18$$
- This demonstrates how Spark can create derived columns without modifying the original dataset.

Current DataFrame Status

The DataFrame now contains the following modifications:

- old_name → new_name
- price converted to **Double**
- final_price added as a new calculated column

These changes will be used in the remaining assignment questions.

Q11. Difference Between Transformations and Actions

Answer

In Apache Spark, operations are classified into **Transformations** and **Actions**.

Transformations

Transformations create a new DataFrame or RDD from an existing one. They are **lazy**, meaning Spark does not execute them immediately. Instead, Spark records these operations and executes them only when an action is called.

Examples:

- filter()
- select()

Actions

Actions trigger the execution of all pending transformations and return a result or write data to storage.

Examples:

- `show()`
- `collect()`

Difference

Transformations	Actions
Lazy execution	Immediate execution
Creates a new DataFrame/RDD	Produces output or saves data
Builds the DAG	Triggers DAG execution
Returns another DataFrame/RDD	Returns a value or writes data

Q12. Load Parquet, Filter Null Values and Save as CSV

Problem Statement

Load a Parquet file, remove records where **user_id** is null, and save the filtered result as a CSV file.

Solution

First, the current DataFrame is saved as a Parquet file. The Parquet file is then loaded, rows with null values in the **user_id** column are removed, and the cleaned data is written as a CSV file.

```
# Save DataFrame as Parquet
df.write.mode("overwrite").parquet("../output/parquet_output")

parquet_df = spark.read.parquet("../output/parquet_output")

filtered_df = parquet_df.filter(
    col("user_id").isNotNull()
)

filtered_df.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("/content/csv_output")

filtered_df.show()
```

Q13. Difference Between Client Mode and Cluster Mode

Answer

Apache Spark supports two deployment modes: **Client Mode** and **Cluster Mode**.

Client Mode

In Client Mode, the **Driver Program** runs on the machine from which the Spark application is

submitted (the client machine), while the executors run on the cluster.

Advantages

- Easy to debug and monitor.
- Suitable for development and testing.

Cluster Mode

In Cluster Mode, both the **Driver Program** and **Executors** run inside the cluster. The Cluster Manager is responsible for managing the Driver and Executors.

Advantages

- Better fault tolerance.
- Suitable for production environments.
- The application continues running even if the client disconnects.

Difference

Client Mode	Cluster Mode
Driver runs on client machine	Driver runs inside the cluster
Easier debugging	Better for production
Client must remain connected	Client can disconnect after submission
Mostly used for development	Mostly used for production

Q14. Filter Dataset Using OR Condition

Problem Statement

Filter the dataset to display records where:

- **region = 'North'**
- **OR**
- **priority = 'High'**

Solution

The following query uses the `filter()` function with the logical **OR (|)** operator.

```
df_q14 = df.filter(  
    (col("region") == "North") |  
    (col("priority") == "High")  
)  
df_q14.show()
```

Observation

- Successfully filtered the dataset using the logical **OR (|)** operator.
- The result contains records where either the **region** is **North** or the **priority** is **High**.

Q15. Why is `.show(5)` Safer than `.collect()`?

Answer

The `.show(5)` function displays only the first five rows of a DataFrame, making it efficient for quickly inspecting data without loading the entire dataset into memory.

On the other hand, `.collect()` retrieves **all rows** from the distributed cluster and transfers them to the Driver program. For very large datasets, this can consume a significant amount of memory and may even cause the Driver to run out of memory or crash.

Advantages of `.show(5)`

- Displays only a few rows.
- Uses minimal memory.
- Suitable for exploring large datasets.
- Faster for previewing data.

Disadvantages of `.collect()`

- Loads the entire dataset into Driver memory.
- High memory consumption.
- Can cause OutOfMemory errors on very large datasets.

Conclusion

For exploratory analysis, `.show(5)` is much safer and more efficient than `.collect()`, especially when working with large-scale data.